Development Report for Issue #300
Tensor-Native Trace Discretization and Skeleton Support

AI-assisted development report

May 29, 2026

1 Purpose of the Work

The purpose of issue #300 was to add the next missing structural layer in the tensor-native stack after the FE-aware volume-only matrix-free path from issue #299. In particular, the goal was to define a tensor-native notion of:

- dimension-specific trace variables,
- tensor-native skeleton support for traces orthogonal to a chosen component dimension, and
- an initial trace-local layout/runtime interpretation compatible with the current tensor-native discretization stack.

At the same time, the issue was intentionally *not* meant to solve full trace-term operator realization. The central design requirement was therefore to introduce the missing topology/discretization/layout/runtime substrate for traces while keeping a clean path toward later side-term realization work.

2 Summary of What Was Implemented

The final implementation added three main pieces of functionality.

2.1 1. Dimension-specific trace semantics on `TensorVar`

`TensorVar` was extended so that trace variables now carry an explicit *trace component ordinal*. Concretely:

- field variables remain `TENSOR_FIELD` and do not carry a trace component ordinal,
- trace variables remain `TENSOR_TRACE` but must now specify the component dimension to which they belong,
- constructor validation was tightened so that field variables cannot accidentally carry trace-dimension metadata, and trace variables cannot omit it, and
- `copyWithNewName(...)` preserves the trace component ordinal for trace variables.

This makes the geometric identity of a trace variable explicit at the variable-descriptor level, which is the natural place for it in the present tensor-native architecture.

2.2 2. Explicit tensor-skeleton support in `TensorMeshTopology`

`TensorMeshTopology` was extended with explicit skeleton support for a chosen orthogonal component dimension. The added API includes:

- `TensorSkeletonEntity`,
- `TensorSkeleton`,
- `retainedComponentOrdinalsForSkeleton(int)`,
- `tensorSkeleton(int)`, and
- `isBoundarySkeletonEntity(...)`.

The implemented semantics are *slice-based*. For a chosen component dimension d, we iterate over active vertices in that component and, for each such vertex, form the slice induced by fixing that vertex. The corresponding tensor-skeleton entities are then the retained active-cell tuples in that slice.

This point is worth stating carefully. Informally, one might describe the geometry as "the component-d skeleton tensored with the remaining component topologies." That remains a useful shorthand. However, the implementation-level interpretation is more precise:

- a boundary vertex in component d typically contributes one retained slice-supported configuration, while
- an interior vertex may contribute multiple retained active-cell tuples because it is incident to active cells on both sides.

Thus the final skeleton support is not merely a cartesian product of vertex count with one retained active-cell tuple per remaining-component configuration; rather, it is the explicit set of retained tuples that arise from the corresponding vertex-fixed slices.

2.3 3. Initial trace-local support in `TensorDofLayout`

`TensorDofLayout` was extended so that the local layout for a registered variable now carries explicit per-component support metadata. In particular:

- a new `TensorComponentSupportKind` enum distinguishes volume-basis support from trace-skeleton support,
- `VariableLayout` now stores per-component support kinds and the distinguished trace component ordinal when applicable, and
- trace variables use a skeleton-support extent in their distinguished component rather than a volume-basis cardinality in that factor.

In the present implementation, the skeleton-support extent is taken to be the active component-vertex extent in the trace component. The remaining components continue to use the same basis-cardinality interpretation already used for volume variables.

This gives a coherent first trace-local coefficient-space model of the form

$$\text{trace-support extent in } d \times \prod_{r \neq d} \text{volume basis extent in } r.$$

Importantly, the resulting local layout remains a deterministic flattening of one variable-local coefficient space, and both variable-ordering and cell-ordering views remain same-size, invertible flattenings of that space.

3 Runtime and Integration Consequences

A useful feature of the final implementation is that the new trace support fit cleanly into the existing tensor-native runtime structure.

3.1 TensorMesh registration

`TensorMesh` did not require a major redesign. The existing variable/discretization registration flow remained adequate once `TensorVar` acquired the missing trace-dimension semantics. Trial and test trace discretizations can now be registered coherently, and the resulting `TensorDofLayout` reflects their trace-local structure.

3.2 TensorGatherScatter compatibility

`TensorGatherScatter` did not require a substantive runtime redesign. Because it is driven by `TensorDofLayout` and remains fundamentally a flat local/global buffer mover, trace compatibility largely followed from the layout changes. Focused tests now verify that trace variables produce coherent local dof counts and remain compatible with gather/scatter round trips.

This is an encouraging architectural result, because it means issue #300 filled the intended structural gap without forcing premature changes in other tensor-native runtime layers.

4 Testing Added or Revised

The implementation added focused tests in the following areas.

4.1 TensorVar tests

The new `TensorVar` tests verify:

- construction of trace variables with an explicit trace component ordinal,
- rejection of missing or out-of-range trace component ordinals,
- continued support for flux-like component vars under `TENSOR_TRACE`, and
- preservation of trace-dimension metadata under `copyWithNewName(...)`.

4.2 TensorMeshTopology tests

The new topology tests verify:

- skeleton entity counts under the final slice-based interpretation,
- deterministic ordering with lower-indexed retained components varying fastest within each vertex slice,
- agreement between skeleton entities and the corresponding `slice(...)` topologies, and

- boundary/interior classification based on component-vertex incidence.

An important stabilization point arose here. An early version of the skeleton-count test assumed the simpler shorthand "vertex count times retained active-cell count," which is not correct at interior vertices under slice-based semantics. The final tests now reflect the intended implementation more accurately by summing the retained active-cell counts of the vertex-fixed slices.

4.3 TensorDofLayout tests

The new layout tests verify:

- support-kind metadata for trace variables,
- use of the component-vertex extent in the distinguished trace component,
- coherent total local dof counts for traces, and
- invertibility of variable-ordering and cell-ordering flattenings for trace-local layouts.

4.4 TensorMesh and TensorGatherScatter integration tests

The integration tests verify:

- registration of trace discretizations on `TensorMesh`,
- creation of trace-aware local layouts through `createDofLayout()`, and
- continued flat-buffer gather/scatter compatibility for trace-local coefficient vectors.

A small test-fix stabilization step was needed here as well: the round-trip gather/scatter test initially used a nonzero-initialized destination buffer even though `scatterLocalToGlobal()` is additive. That expectation was corrected, and the final tests now reflect the intended additive-scatter semantics.

5 Final Geometric Interpretation

The final implemented interpretation of trace support in the current orthogonal 1D component setting may be summarized as follows.

Let d be the distinguished trace component dimension. Then:

1. the component-d skeleton is represented by the active vertices in that component,
2. each such vertex defines a lower-dimensional tensor slice through the active tensor topology, and
3. the tensor-native trace-support entities are the retained active-cell tuples of those slices.

Equivalently, one may think of traces as living on the skeleton orthogonal to dimension d, with the understanding that interior component vertices induce multiple slice-supported retained tuples. This is the precise structural model that was implemented and tested.

6 What Issue #300 Did *Not* Attempt

It is equally important to record what was intentionally left for later work.

Issue #300 did *not* implement:

- trace-aware basis evaluation on the new skeleton entities,
- side quadrature or lower-dimensional physical-point generation for trace domains,
- neighbor pairing or orientation logic for interior trace terms,
- mixed volume/trace local kernels, or
- trace-aware matrix-free or PAMatrix-backed operator realization.

Those topics remain the natural next step, but the present work now gives them a well-defined tensor-native support model to build on.

7 Overall Assessment

From an architectural perspective, issue #300 achieved its intended purpose. The tensor-native stack now has:

- trace variables with explicit component-dimension identity,
- explicit tensor-native skeleton support for traces orthogonal to a chosen component dimension,
- an initial trace-local coefficient/layout interpretation, and
- focused tests that pin down the resulting semantics.

Just as importantly, this was accomplished without disrupting the current tensor-native descriptor/discretization/runtime layering. In particular, the work fit naturally into `TensorVar`, `TensorMeshTopology`, `TensorDofLayout`, `TensorMesh`, and `TensorGatherScatter`, which suggests that the underlying design direction remains sound.

8 Next Step

The next natural task is to make use of this new support in operator realization. In particular, the most direct follow-on is to implement tensor-native trace-aware local operator realization on the new skeleton substrate. That work should address:

- trace-local basis interpretation,
- lower-dimensional quadrature and physical-point generation on tensor skeletons,
- representative mixed volume/trace and trace/trace terms, and
- the corresponding integration of those terms into the existing tensor-native matrix-free operator path.

That is the natural place where the structural work of issue #300 can begin to pay off numerically.